SECTION 9 · CHEAT SHEET

Barista Bot: your own agent with the SDK

Every prompt, command, shortcut and setting from this section, lesson by lesson. Prompts are word for word, as typed on screen.

9.1 Section 9 introduction

Concept lesson: nothing to type, run or set.

9.2 When to build your own agent

RESULT — App and command line
For your own work: the agent is Google's; you pick the model, rules and skills.

RESULT — Antigravity SDK (Python)
When the agent is the product: your code starts it and sets its voice, tools and which actions wait for a person.

9.3 Google AI Studio and your API key

SETTING — AI Studio › Dashboard › API Keys › Create API key
A name and a project. The key belongs to the project; the project's tier sets the limits.

SETTING — AI Studio › Dashboard › Rate Limit
Requests per minute and per day. One question to an agent can take several requests.

FILE — barista-bot/.env
GEMINI_API_KEY= your key. Listed in .gitignore; never in a prompt, a screenshot or Git.

9.4 Let the agent write your agent

PROMPT

```
Build the Barista Bot: a small agent of the cafe's own, written with Google's
Antigravity SDK, in a new folder called barista-bot. Give it its own virtual
environment and a requirements.txt that pins google-antigravity==0.1.23. It answers
customer questions only from baseline/menu.csv and baseline/cafe_faq.md, and when
the answer isn't in those files it says so politely instead of guessing. It reads
GEMINI_API_KEY from barista-bot/.env: create a .env.example, add .env and the
virtual environment to .gitignore, and never ask me for the key in chat, I will put
the real key in .env myself. Keep it a simple terminal chat: python bot.py, until
the customer types quit. Install it, check that it starts, and tell me how to run
it.
```

bot.py, requirements.txt, .env.example, its own .venv. Put the real key into .env yourself.

COMMAND

```
.\barista-bot\.venv\Scripts\python barista-bot\bot.py
```

Answers come from the menu and the FAQ; a question they can't answer gets a polite no. **quit** ends it.

9.5 Personality and tools

PROMPT

```
Give the Barista Bot a personality and two tools. Personality, as its system
instructions: a warm, cheerful barista at KGP Vibes Cafe who keeps answers short,
greets regulars kindly, and never makes up items or prices. Tools, as plain Python
functions passed to the agent: menu_lookup(name), which returns an item's row from
baseline/menu.csv (price, diet, whether it contains egg), and
suggest_pairing(item), which returns the pairs_with item from the same file, so
pairings come from the menu and are never invented. Keep everything else in bot.py
the same, and tell me how to try it.
```

Read the review: red lines showed the grounding rules and house rules replaced.

PROMPT

```
You replaced the whole system instructions, so the bot lost its grounding rules,
the house rules, the menu and the FAQ. Put all of that back exactly as it was, and
add the barista personality on top of it. Keep the two tools as they are.
```

The second review is green: the rules are back, the persona sits on top.

FILE

barista-bot/bot.py

LocalAgentConfig(..., tools=[menu_lookup, suggest_pairing]): a tool is a plain function.

9.6 Show it a photo

PROMPT

```
Teach the Barista Bot to read a photo. I put today's specials board in barista-
bot/specials_board.png. When a customer types photo, the bot sends that image to
the agent along with their question, so it can answer things like what's special
today and is it vegan. Use the SDK's image input, keep everything else the same,
and tell me how to try it.
```

The grounding rules name the photo as a third source.

FILE barista-bot/bot.py

[Image.from_file("specials_board.png"), user_prompt]: the picture and the question in one message.

RESULT photo What's special today, and is any of it vegan?

Four specials and their prices, read off the board; the vegan one named.

9.7 Keep a human in charge

PROMPT

```
Add a discount tool to the Barista Bot, kept behind the owner's OK.
apply_discount(item, percent) is a plain Python function that only prints the
discounted price; it changes nothing on disk. Use the SDK's policy so the agent
must ask the owner in the terminal before apply_discount runs, and if the owner
says no, the bot tells the customer politely. Keep everything else the same, and
tell me how to try it.
```

A handler that waits for y or n, and a policy that calls it.

FILE barista-bot/bot.py

policies=[policy.ask_user("apply_discount", handler=ask_owner_approval), *policy.confirm_run_command()]

RESULT Can I get 20% off a Tech Latte?

The approval line appears; **n** gives a polite no, **y** lets the tool run.

9.8 Run it on your own machine

PROMPT

```
Let the Barista Bot also run on a local model. Add a --local flag to bot.py: with
it, the agent uses Ollama on this computer at http://localhost:11434/v1 with the
model gemma4:31b through the SDK's LocalOpenAIAgentConfig; without it, everything
stays on Gemini as before. Don't list or pull any Ollama models. Tell me how to try
both.
```

The tools and policies move into shared lists; one if picks the config.

FILE barista-bot/bot.py

LocalOpenAIAgentConfig(model="gemma4:31b", base_url="http://localhost:11434/v1", ...): no key needed.

COMMAND

```
.\barista-bot\.venv\Scripts\python barista-bot\bot.py --local
```

Same answers from the same files, nothing sent out. Only as fast as your hardware.

RESULT LiteRT

For phones and laptops: smaller models on the device itself, with its own SDK config.